

Backpropagation Through Time (BPTT)

1. Why We Need Backpropagation in RNNs

RNNs learn patterns from **sequential data** by updating their **weights** over time — just like normal neural networks do using **backpropagation**.

But RNNs are special because:

- The **same weights** are reused at every time step.
- The **output at time t** depends on **previous hidden states**.

So, normal backpropagation isn't enough, we must apply it **through time**.

That's why we use **BPTT = Backpropagation Through Time**.

2. How an RNN Works (Forward Pass)

At each time step t , the RNN does this:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Where:

- x_t = input at time t
- h_t = hidden state at time t
- y_t = output
- W_{xh}, W_{hh}, W_{hy} = weights (input → hidden, hidden → hidden, hidden → output)

The **same weights** are shared across all time steps — that's what makes it "recurrent".

3. Why "Through Time"?

Imagine you have a sequence:

[x_1, x_2, x_3, x_4]

The RNN unfolds through time like this:

$x_1 \rightarrow h_1 \rightarrow y_1$

$x_2 \rightarrow h_2 \rightarrow y_2$

$x_3 \rightarrow h_3 \rightarrow y_3$

$x_4 \rightarrow h_4 \rightarrow y_4$

Each hidden state depends on the previous one:

$$h_2 = f(W_{xh} * x_2 + W_{hh} * h_1)$$

$$h_3 = f(W_{xh} * x_3 + W_{hh} * h_2)$$

...

So, when we calculate the **loss** at the end (say at time $t=4$), that loss depends on all previous hidden states — because information flowed from $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4$.

4. What Is Backpropagation Through Time (BPTT)?

BPTT means we:

1. **Unfold** the RNN across all time steps ($t=1$ to T)
2. **Apply backpropagation** on this unfolded network (like a deep feedforward network)
3. **Sum gradients** across all time steps because weights are shared

Visually:

Forward: $x_1 \rightarrow h_1 \rightarrow y_1 \rightarrow x_2 \rightarrow h_2 \rightarrow y_2 \rightarrow x_3 \rightarrow h_3 \rightarrow y_3 \rightarrow \dots$

Backward: $\leftarrow \text{-----Backpropagation Through Time-----} \leftarrow$

5. The Gradient Flow (Step-by-Step)

Let's say total loss = sum of all time step losses:

$$L = \sum_{t=1}^T L_t$$

To update weights like W_{hh} , we need:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_{hh}}$$

But — here's the tricky part —

L_t depends not only on h_t , but also on $h_{t-1}, h_{t-2}, \dots, h_1$

So, we must backpropagate **through all previous time steps**.

That's why it's called **Backpropagation Through Time**.

6. Gradient Computation Intuition

At each step:

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_{hh}}$$

But h_t depends on h_{t-1} , which depends on h_{t-2} , and so on...

So, the chain rule expands backward in time:

$$\frac{\partial L}{\partial W_{hh}} = \sum_t \left(\frac{\partial L_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdot \dots \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_{hh}} \right)$$

7. Vanishing and Exploding Gradients

Here's the main challenge with BPTT:

Each step multiplies gradients by the derivative of the activation (like tanh or sigmoid).

If derivatives $< 1 \rightarrow$ gradients **shrink** exponentially $\rightarrow$ **Vanishing gradient**

If derivatives $> 1 \rightarrow$ gradients **explode** exponentially $\rightarrow$ **Exploding gradient**

As a result:

- Early time steps get **tiny gradients** (network forgets long-term info)
- Training becomes unstable or very slow

8. How to Fix These Problems

Common strategies:

1. **Gradient Clipping** — Limit gradient values to avoid explosion.
2. **Use better architectures:**
 - **LSTM (Long Short-Term Memory)**
 - **GRU (Gated Recurrent Unit)**These were designed specifically to **preserve long-term dependencies**.
3. **Use ReLU or leaky activations**
4. **Shorter sequences / Truncated BPTT** — Instead of backpropagating through the entire sequence, stop after a few steps.

9. Truncated BPTT (Practical Version)

When sequences are long (like 1000 time steps), backpropagating through all of them is too costly.

So we use **Truncated BPTT**:

- Split the sequence into smaller chunks (e.g., 10 steps)
 - Backpropagate only within each chunk
 - Hidden states are passed between chunks, but gradients are not
1. Faster
 2. More memory-efficient
 3. Still captures temporal relationships

10. Intuitive Summary

Concept	Meaning
Forward pass	Process input sequence and compute outputs
Backward pass (BPTT)	Unroll network through time and propagate error backward
Shared weights	Same parameters used at every time step
Vanishing gradient	Early steps receive almost no learning signal
Exploding gradient	Gradient becomes huge and training diverges
Truncated BPTT	Limit backprop steps to save memory and time

➤ Code Example — Keras

Keras automatically handles BPTT when training an RNN:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(64, input_shape=(timesteps, features)),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy')
model.fit(X_train, y_train, epochs=10)
```

Behind the scenes, TensorFlow performs **Backpropagation Through Time** automatically.

➤ Easy Analogy

Imagine reading a story:

- Each sentence adds to your memory.
- At the end, you realize you misunderstood something in the middle.

- You mentally “**backtrack**” through earlier sentences to fix your understanding. That’s exactly what **BPTT** does — it looks back through previous time steps to correct earlier weights.

Final Takeaways

- **BPTT** = Backpropagation applied across time steps.
 - It updates **shared weights** using accumulated gradients.
 - It suffers from **vanishing/exploding gradients** → solved by **LSTM/GRU**.
 - **Truncated BPTT** is the practical solution for long sequences.
-